

Backend Golang

Test v1.5.0

Ketentuan :

1. Waktu pengerjaan max 1 x 24 jam, terhitung dari sejak anda mendapatkan test ini
2. Jawaban & code snippet bisa anda sertakan di platform seperti google doc/github atau lainnya, yang terutama bisa diakses publik, mohon sertakan link jawaban anda.
3. Khusus untuk jawaban coding test (simple project test) harus dikerjakan pada git repository dan bisa diakses publik.

I. Test pengetahuan Golang

1. Framework golang apa saja yang pernah anda pakai? Dan jelaskan secara singkat dengan framework tersebut project apa yang anda kerjakan dan mengapa menggunakan framework tersebut.
2. Jelaskan apa yang anda ketahui mengenai concurrency pada golang?
3. Buatlah contoh kode/function golang yang memanfaatkan antara WaitGroup dan Channel, dan jelaskan kapan bisa menggunakan waitgroup atau channel atau keduanya secara bersamaan.
4. Apa yang anda ketahui dengan goroutine? Pernahkah memakainya? Ceritakan jika pernah, jelaskan kegunaan dan tujuan nya dalam project yang pernah anda kerjakan.
5. Jelaskan apa yang anda ketahui mengenai queueing pada golang, dan buat contoh kode sederhana.
6. Dalam microservices yang semuanya menggunakan Go, bagaimana cara/metode terbaik masing2 service berkomunikasi satu sama lain?
7. Buatlah contoh kode dalam Go, yang menggambarkan komunikasi antar service yang telah anda jelaskan diatas.
8. Bagaimana anda mengatasi error dalam Go? Ceritakan pengalaman anda.
9. Apakah anda memiliki pengalaman dalam error logging dalam Go? Jika ya, jelaskan bagaimana anda menyimpan atau menampilkan setiap log agar mudah dikelola, terutama dalam microservices.

10. Buat kode dalam golang, untuk output perkiraan ongkos kirim suatu order jika diketahui:

- Kurir bernama X Kargo, pengiriman dari Gudang A ke Kota B adalah Rp.10.000,- per kg
- Data pesanan:

Nama Produk	Berat Bruto (gram)	Panjang (mm)	Lebar (mm)	Tinggi (mm)
Produk A	30	115	85	25
Produk B	28000	1290	300	625

Hal yang perlu diperhatikan:

- a. Function bisa dibuat dengan fungsi umum atau fungsi closure (named / unnamed function)
- b. Code yang dibuat harus bisa mengkalkulasi kondisi jumlah produk yang dinamis, contoh : Produk A quantity n, Produk B quantity m
- c. Penentuan atau kalkulasi berat suatu barang berdasarkan berat aktual dan volumetrik.

11. Ceritakan kesulitan yang sering anda temui saat anda mengerjakan project dengan bahasa Go. Dan bagaimana anda mengatasi kesulitan tersebut.

II. Coding test 1 (Go)

Project test ini dikerjakan pada git (github/gitlab/bitbucket), dan sertakan file go build dan env jika diperlukan.

Concurrent Data Aggregation Service

Penjelasan:

Buat sebuah service yang dapat secara concurrent mengambil data untuk “ItemDetails” yang berasal dari simulasi API eksternal, service ini harus mampu membatasi jumlah request ke API eksternal dan mengelola timeout serta error dari setiap request individu.

Struktur Data:

```
// ItemDetails
type ItemDetails struct {
    ID      string
    Name    string
    Description string
    Price   float64
}
```

Simulasi API Eksternal:

Gunakan fungsi `simulateFetchItemDetails` di bawah ini. Fungsi ini mensimulasikan panggilan API eksternal yang mengimplementasi :

1. Random latency antara 500ms hingga 2000ms.
2. kemungkinan 15% untuk return error.
3. Respon `context.Context` untuk cancellation.

```
package main

import (
    "context"
    "fmt"
    "log"
    "math/rand"
    "sync"
    "time"
)

// ItemDetails represents the detailed information for an item.
```

```

type ItemDetails struct {
    ID      string
    Name     string
    Description string
    Price    float64
}

// simulateFetchItemDetails simulates an external API call to fetch item details.
// It introduces random delays and occasional errors.
// It respects the context for cancellation.
func simulateFetchItemDetails(ctx context.Context, itemID string) (*ItemDetails, error) {
    // Simulate network latency
    delay := time.Duration(500+rand.Intn(1500)) * time.Millisecond // 0.5s to 2s
    select {
    case <-time.After(delay):
        // Continue
    case <-ctx.Done():
        log.Printf("Context cancelled for item %s, aborting fetch.", itemID)
        return nil, ctx.Err() // Context cancelled
    }

    // Simulate occasional API errors
    if rand.Intn(100) < 15 { // 15% chance of error
        log.Printf("Simulated API error for item %s", itemID)
        return nil, fmt.Errorf("simulated API error for item %s: service unavailable", itemID)
    }

    details := &ItemDetails{
        ID:      itemID,
        Name:     fmt.Sprintf("Product %s", itemID),

```

```

        Description: fmt.Sprintf("Detailed description for product %s.", itemID),
        Price:      rand.Float64() * 100,
    }
    log.Printf("Successfully fetched details for item %s", itemID)
    return details, nil
}

```

Yang harus anda kerjakan :

Implementasikan fungsi FetchAndAggregate dengan menggunakan parameter berikut:

```

func FetchAndAggregate(
    ctx context.Context,
    itemIDs []string,
    maxConcurrent int,
    perItemTimeout time.Duration,
) (map[string]ItemDetails, []error)

```

Persyaratan:

1. Concurrency: Fungsi harus secara concurrent memanggil simulateFetchItemDetails untuk setiap itemID dalam itemIDs.
2. Batas Concurrency (maxConcurrent): Tidak boleh ada lebih dari maxConcurrent request simulateFetchItemDetails yang berjalan pada waktu yang bersamaan, untuk mencegah overloading API eksternal.
3. Timeout Per Item (perItemTimeout): Setiap panggilan simulateFetchItemDetails harus memiliki timeout sendiri. Jika request untuk satu item melebihi perItemTimeout, request tersebut harus dibatalkan, dan dianggap sebagai error untuk item tersebut.
4. Penanganan Error:

- a. Fungsi tidak boleh gagal total jika ada satu atau beberapa request item yang gagal (karena error API atau timeout).
 - b. Semua ItemDetails yang berhasil diambil harus direturn dalam map[string]ItemDetails dengan kunci adalah ID item.
 - c. Semua error dari request item yang gagal harus di collect dan di return sebagai []error.
5. context.Context Global: Fungsi FetchAndAggregate harus implement ctx yang diterimanya. Jika ctx cancelled (misalnya, karena timeout global atau shutdown), semua proses simulateFetchItemDetails yang sedang berjalan harus langsung dibatalkan.

II. Coding test 2 (Go)

Project test ini dikerjakan pada git (github/gitlab/bitbucket), dan sertakan file go build dan env jika diperlukan.

Distributed Task Processing with State Management

Description :

Buat sebuah service Go yang memproses "report". Service ini terdiri dari dua komponen utama:

1. Data Requester (Producer): function yang mensimulasikan request pembuatan report dan mengirim ke RabbitMQ.
2. Data Processor (Consumer/Worker Pool): service yang mengambil request report dari RabbitMQ, memproses secara concurrent, dan update status serta simpan hasil proses report di Redis.

Prasyarat :

- Gunakan context.Context untuk cancellation dan timeout di seluruh sistem.
- Implementasi graceful shutdown untuk semua komponen (producer dan consumer) saat menerima sinyal SIGINT (Ctrl+C).
- Error handling harus robust, termasuk kemungkinan gagal koneksi ke RabbitMQ/Redis dan kegagalan dalam pemrosesan laporan.
- Pastikan message acknowledgement di RabbitMQ dilakukan pada waktu yang

tepat.

- Pastikan concurrency worker dapat dibatasi.
- Dependency :
 - go get github.com/streadway/amqp
 - go get github.com/go-redis/redis/v8
- Struktur file (optional) :

```
golang-test-two/  
├─ main.go  
└─ (optional) internal/  
    ├─ config/  
    │   └─ config.go  
    ├─ models/  
    │   └─ models.go  
    └─ services/  
        ├─ rabbitmq.go  
        ├─ redis.go  
        └─ processor.go
```

- Definition dan struktur :

```
package main  
  
import (  
    "context"  
    "encoding/json"  
    "fmt"  
    "log"  
    "math/rand"  
    "os"  
    "os/signal"  
    "sync"  
    "syscall"  
    "time"  
  
    "github.com/go-redis/redis/v8"  
    "github.com/streadway/amqp"  
)  
  
// --- Konfigurasi ---  
const (  
    RABBITMQ_URL = "amqp://guest:guest@localhost:5672/"  
    REDIS_URL    = "localhost:6379"
```

```

QUEUE_NAME = "report_requests"
// KEY_PREFIX_REPORT_STATUS digunakan untuk menyimpan status laporan di Redis
KEY_PREFIX_REPORT_STATUS = "report:status:"
// KEY_PREFIX_REPORT_DATA digunakan untuk menyimpan data hasil laporan di Redis
KEY_PREFIX_REPORT_DATA = "report:data:"

NUM_PRODUCER_REQUESTS = 10 // Jumlah permintaan laporan yang akan dibuat
NUM_WORKERS = 3 // Jumlah worker konkruen untuk memproses laporan
WORKER_TIMEOUT = 5 * time.Second // Timeout per worker untuk setiap tugas
PUBLISH_INTERVAL = 500 * time.Millisecond // Interval producer mengirim pesan
)

// --- Struktur Data ---

// ReportRequest mewakili permintaan untuk membuat laporan.
type ReportRequest struct {
    ID string `json:"id"`
    ReportType string `json:"report_type"` // e.g., "sales", "inventory"
    Parameters map[string]string `json:"parameters"`
    CreatedAt time.Time `json:"created_at"`
}

// ReportStatus mewakili status pemrosesan laporan.
type ReportStatus string

const (
    StatusPending ReportStatus = "PENDING"
    StatusInProgress ReportStatus = "IN_PROGRESS"
    StatusCompleted ReportStatus = "COMPLETED"
    StatusFailed ReportStatus = "FAILED"
)

// ReportResult mewakili hasil akhir dari laporan yang diproses.
type ReportResult struct {
    RequestID string `json:"request_id"`
    Status ReportStatus `json:"status"`
    GeneratedAt time.Time `json:"generated_at"`
    ReportData string `json:"report_data,omitempty"` // Contoh data laporan (bisa complex struct)
    Error string `json:"error,omitempty"`
}

// --- Fungsi Pembantu (Untuk Simulasi dan Koneksi) ---

// connectRabbitMQ establishes a connection and channel to RabbitMQ.
func connectRabbitMQ(ctx context.Context) (*amqp.Connection, *amqp.Channel, error) {
    conn, err := amqp.Dial(RABBITMQ_URL)
    if err != nil {
        return nil, nil, fmt.Errorf("failed to connect to RabbitMQ: %w", err)
    }

    ch, err := conn.Channel()
    if err != nil {
        conn.Close()
        return nil, nil, fmt.Errorf("failed to open a channel: %w", err)
    }

    _, err = ch.QueueDeclare(
        QUEUE_NAME, // name
        true, // durable
        false, // delete when unused

```



```

        false, // exclusive
        false, // no-wait
        nil,   // arguments
    )
    if err != nil {
        ch.Close()
        conn.Close()
        return nil, nil, fmt.Errorf("failed to declare a queue: %w", err)
    }

    log.Println("Successfully connected to RabbitMQ and declared queue:", QUEUE_NAME)
    return conn, ch, nil
}

// connectRedis establishes a connection to Redis.
func connectRedis(ctx context.Context) *redis.Client {
    rdb := redis.NewClient(&redis.Options{
        Addr: REDIS_URL,
        DB: 0, // use default DB
    })

    // Use the provided context for the Ping operation
    err := rdb.Ping(ctx).Err()
    if err != nil {
        log.Fatalf("Failed to connect to Redis: %v", err)
    }
    log.Println("Successfully connected to Redis:", REDIS_URL)
    return rdb
}

// simulateReportGeneration simulates the actual report generation process.
// It includes random delays and a chance of failure.
// It also respects its own context for timeout/cancellation.
func simulateReportGeneration(ctx context.Context, request ReportRequest) (string, error) {
    log.Printf("[Worker: %s] Starting report generation for ID: %s (Type: %s)", request.ID, request.ID,
        request.ReportType)

    // Simulate CPU-intensive work or external API calls
    delay := time.Duration(1 + rand.Intn(5)) * time.Second // 1 to 5 seconds
    select {
    case <-time.After(delay):
        // Continue processing
    case <-ctx.Done():
        log.Printf("[Worker: %s] Context cancelled during simulation for ID: %s", request.ID, request.ID)
        return "", ctx.Err() // Propagate context cancellation error
    }

    // Simulate random failure (e.g., database error, invalid parameters)
    if rand.Intn(100) < 20 { // 20% chance of failure
        log.Printf("[Worker: %s] Simulated failure for ID: %s", request.ID, request.ID)
        return "", fmt.Errorf("simulated report generation error for ID %s", request.ID)
    }

    reportData := fmt.Sprintf("Report %s - Type: %s, Generated On: %s, Data: Random Value %d",
        request.ID, request.ReportType, time.Now().Format(time.RFC3339), rand.Intn(1000))

    log.Printf("[Worker: %s] Successfully generated report for ID: %s", request.ID, request.ID)
    return reportData, nil
}

```

Yang harus anda kerjakan :

Implementasikan fungsi-fungsi berikut untuk membangun sistem pemrosesan laporan:

1. *produceReportRequests(ctx context.Context, wg *sync.WaitGroup):*
 - Fungsi ini akan membuat NUM_PRODUCER_REQUESTS permintaan report.
 - Setiap permintaan akan di-marshal ke JSON dan dipublikasikan ke antrian RabbitMQ (QUEUE_NAME).
 - **Note:** Pastikan pesan durable (amqp.Persistent).
 - Fungsi ini harus berhenti saat ctx dibatalkan.
2. *updateReportStatus(ctx context.Context, rdb *redis.Client, requestID string, status ReportStatus, reportData string, errMsg string):*
 - Fungsi ini bertanggung jawab untuk memperbarui status report di Redis.
 - Jika status adalah StatusCompleted atau StatusFailed, ReportResult lengkap harus disimpan di KEY_PREFIX_REPORT_DATA:{requestID}.
 - Untuk semua status, status saat ini harus disimpan di KEY_PREFIX_REPORT_STATUS:{requestID}.
 - **Note:** Gunakan json.Marshal dan rdb.Set atau rdb.HSet untuk menyimpan data JSON di Redis. Gunakan context untuk operasi Redis.
3. *reportWorker(ctx context.Context, workerID int, tasks <-chan amqp.Delivery, results chan<- ReportResult, rdb *redis.Client):*
 - NUM_WORKERS goroutine akan menjalankan fungsi ini.
 - Setiap worker akan menerima amqp.Delivery dari channel tasks.
 - Untuk setiap amqp.Delivery:
 - Un-marshal pesan menjadi ReportRequest.
 - Segera perbarui status report di Redis menjadi IN_PROGRESS (updateReportStatus).
 - Panggil simulateReportGeneration dengan context turunan yang memiliki WORKER_TIMEOUT.
 - Berdasarkan hasil simulateReportGeneration:
 - Jika sukses, perbarui status menjadi COMPLETED dan simpan ReportData.
 - Jika gagal (karena simulasi error atau timeout), perbarui status menjadi FAILED dan simpan Error.
 - Kirim ReportResult ke channel results.
 - **Note:** Worker ini tidak melakukan Ack atau Nack. Itu akan ditangani oleh komponen lain.
 - Fungsi ini harus berhenti saat ctx dibatalkan atau channel tasks ditutup.

4. *resultAckHandler(ctx context.Context, results <-chan ReportResult, ch *amqp.Channel, rdb *redis.Client, deliveries map[string]amqp.Delivery):*
- Fungsi ini berjalan sebagai goroutine terpisah.
 - Menerima ReportResult dari channel results.
 - Berdasarkan ReportResult.Status:
 - Jika StatusCompleted atau StatusFailed: Lakukan d.Ack(false) untuk amqp.Delivery yang sesuai.
 - Jika StatusFailed karena transient error (contoh: timeout), pertimbangkan d.Nack(false, true) untuk me-requeue pesan, atau d.Nack(false, false) jika fatal error. Untuk soal ini, anggap semua kegagalan simulateReportGeneration adalah fatal (tidak perlu di-requeue).
5. *startReportProcessor(ctx context.Context, wg *sync.WaitGroup, rdb *redis.Client):*
- Fungsi ini adalah orkestrator untuk sisi consumer.
 - Menghubungkan ke RabbitMQ dan mendaftar sebagai consumer (ch.Consume).
 - **Note:** Set autoAck ke false saat memanggil ch.Consume untuk mengaktifkan manual acknowledgement.
 - Membuat channel tasks (untuk distribusi ke worker) dan results (untuk menerima hasil dari worker).
 - Memulai NUM_WORKERS goroutines reportWorker.
 - Memulai resultAckHandler goroutine.
 - Loop utama untuk menerima amqp.Delivery dari RabbitMQ:
 - Un-marshall pesan menjadi ReportRequest.
 - Segera update status laporan di Redis menjadi PENDING (updateReportStatus).
 - Kirim amqp.Delivery ke channel tasks.
 - Challenge: Bagaimana resultAckHandler bisa mengetahui amqp.Delivery mana yang harus di-Ack atau Nack? Anda perlu menyimpan mapping requestID ke amqp.Delivery yang relevan. map[string]amqp.Delivery yang diakses secara thread-safe adalah jawabannya.
 - Fungsi ini harus berhenti saat ctx dibatalkan.
 - Pastikan semua worker dan resultAckHandler selesai sebelum fungsi ini selesai.
-

